# Practice Set: Templates in C++

## Section 1: Conceptual & Practical Questions (1–10)

**Question 1:** Define a **Template** in C++ and explain its primary purpose in programming. How does using templates prevent writing redundant code for different data types like int, float, and double?

**Question 2:** What does the term "generalize" mean in the context of C++ templates? Explain the basic syntax used to declare a template using both typename and class keywords.

**Question 3:** What is a **Placeholder** in C++ template syntax? What character is typically used as a placeholder, and can it be named custom identifier names?

**Question 4:** Name and define the two main types of templates available in C++.

**Question 5:** What is a **Function Template** in C++? Write the syntax and code structure of a simple function template add() that takes two values of a generic type T and returns their sum.

**Question 6:** Explain how the C++ compiler internally handles implicit function template instantiation when calls like add(2, 3) and add(2.5, 3.5) are encountered in the code.

**Question 7:** Explain how to perform an **Explicit Template Call** in C++. How does the syntax for explicit instantiation differ from implicit function template calls?

**Question 8:** How can you define a function template that accepts parameters of two **different data types** (e.g., an int and a float)? Provide the syntax and a brief code example showing show(10, 2.5).

**Question 9:** What is a **Class Template** in C++? Write a complete C++ program demonstrating a generic Box class template that stores a value of type T and displays it using a show() member function.

**Question 10:** How do you instantiate an object of a class template in C++? Show the exact syntax required to create a Box object for int data and another Box object for float data.

## Section 2: Interview & Viva Questions (11–20)

**Question 11 (Viva):** What keyword is used to initiate a template declaration in C++?

**Question 12 (Viva):** Are the keywords typename and class interchangeable inside template parameter lists like template <class T>?

**Question 13 (Viva):** What are the two primary types of templates in C++?

**Question 14 (Viva):** What process does the compiler perform internally when it generates type-specific function versions from a function template at compile time?

**Question 15 (Viva):** How do you pass multiple generic types to a single template declaration?

**Question 16 (Viva):** What is the syntax for explicitly specifying the template argument type when calling a template function add for type double?

**Question 17 (Viva):** When creating an object from a class template Box, is providing the explicit type in angle brackets (e.g., Box<int>) mandatory in C++98/C++03?

**Question 18 (Viva):** Can templates be used to write a single function that prints both a string and an integer simultaneously?

**Question 19 (Viva):** What is the main advantage of using templates over writing multiple overloaded functions for different data types?

**Question 20 (Viva):** In a class template, does member function execution depend on the data type specified during object instantiation?

## Answers & Explanations

### Answer: 01

**Definition:** A template allows us to write one piece of code that works with different data types.

**Purpose & Redundancy Prevention:** Instead of manually writing the same code repeatedly for int, float, double, etc., we write it once as a generic blueprint, and C++ automatically generates the correct type-specific version.

### Answer: 02

**Generalize:** In templates, "to generalize" means creating code that operates on generic data types rather than being hardcoded to specific primitives.

**Syntax:**

template <typename placeholder>// OR template <class placeholder>
Both typename and class keywords are used to define generic placeholders.

### Answer: 03

**Placeholder:** A placeholder variable inside angle brackets <> represents a generic type that will be replaced by an actual data type when instantiated.

**Naming:** It is generally named T, but it can be any valid custom identifier name.

### Answer: 04

**Function Template:** A template declaration followed by a function definition.

**Class Template:** A template declaration followed by a class definition.

### Answer: 05

**Function Template:** A function prefixed with a template <...> header, allowing the function to accept generic parameters and return generic types.

**Code Example:**

```
#include <iostream>
using namespace std;
template <typename T>
T add(T a, T b)
{
   return a + b;
}
int main()
{
   cout << add(2, 3) << endl;     // Calls add for int
   cout << add(2.5, 3.5) << endl; // Calls add for double
   return 0;
}
```

### Answer: 06

When the compiler encounters implicit function template calls, it automatically deduces the type and generates specific function definitions internally:

For add(2, 3), the compiler converts and generates: int add(int, int).

For add(2.5, 3.5), the compiler converts and generates: double add(double, double).

### Answer: 07

**Explicit Template Call:** Explicitly specifying the required data type inside angle brackets <> during a function call rather than letting the compiler deduce it.

**Syntax Comparison:**

*Implicit Call:* add(2, 3);

*Explicit Call:* add<int>(2, 3); or add<double>(2.5, 3.5);

### Answer: 08

Use multiple placeholders inside the template header, such as template <typename T1, typename T2>.

**Code Example:**

```
#include <iostream>

using namespace std;

template <typename T1, typename T2>
void show(T1 a, T2 b)
{
   cout << a << " " << b << endl;
}
int main()
{
   show(10, 2.5);      // T1 = int, T2 = double
   show("Hello", 5);    // T1 = const char*, T2 = int
   return 0;
}
```

### Answer: 09

**Class Template:** A generic class definition prefixed by template <class T> or template <typename T>, allowing its member variables and functions to work with generic types.

**Code Example:**

```
#include <iostream>
using namespace std;
template <class T>class Box
{
   T value;public:
   Box(T v) {
      value = v;
   }
   void show()
   {
      cout << value << endl;
   }
};
int main()
{
```

```
    Box<int> b1(10);
    Box<float> b2(3.14);

    b1.show();
    b2.show();

    return 0;
}
```

### Answer: 10

**Syntax:** ClassName<DataType> objectName(arguments)

**Example:**

Box<int> b1(10);  // Instantiates Box for integer operationsBox<float> b2(3.14);
// Instantiates Box for float operations

### Answer: 11

The **template** keyword.

### Answer: 12

**Yes**, both typename and class can be used interchangeably to declare generic placeholders in template headers.

### Answer: 13

**Function Template** and **Class Template**.

### Answer: 14

**Template Instantiation** (where the compiler automatically generates concrete function definitions like int add(int, int)).

### Answer: 15

By separating placeholder names with commas inside angle brackets: template <typename T1, typename T2>.

### Answer: 16

**add<double>(2.5, 3.5);**

### Answer: 17

**Yes**, class templates require explicit specification of the target data type inside angle brackets during object instantiation (e.g., Box<int> b1(10);).

### Answer: 18

**Yes**, by defining a multi-type function template like template <typename T1, typename T2> void show(T1 a, T2 b) and passing "Hello" and 5.

### Answer: 19
It eliminates redundant code duplication across multiple data types, allowing a single generic code block to be maintained.

### Answer: 20
**Yes**, the data type specified during object instantiation (e.g., <int> or <float>) determines the type of member variables and parameter processing inside the class template.